



AVL (Adelson-Velskii and Landis) Trees

Kuliah Algoritma dan Struktur Data

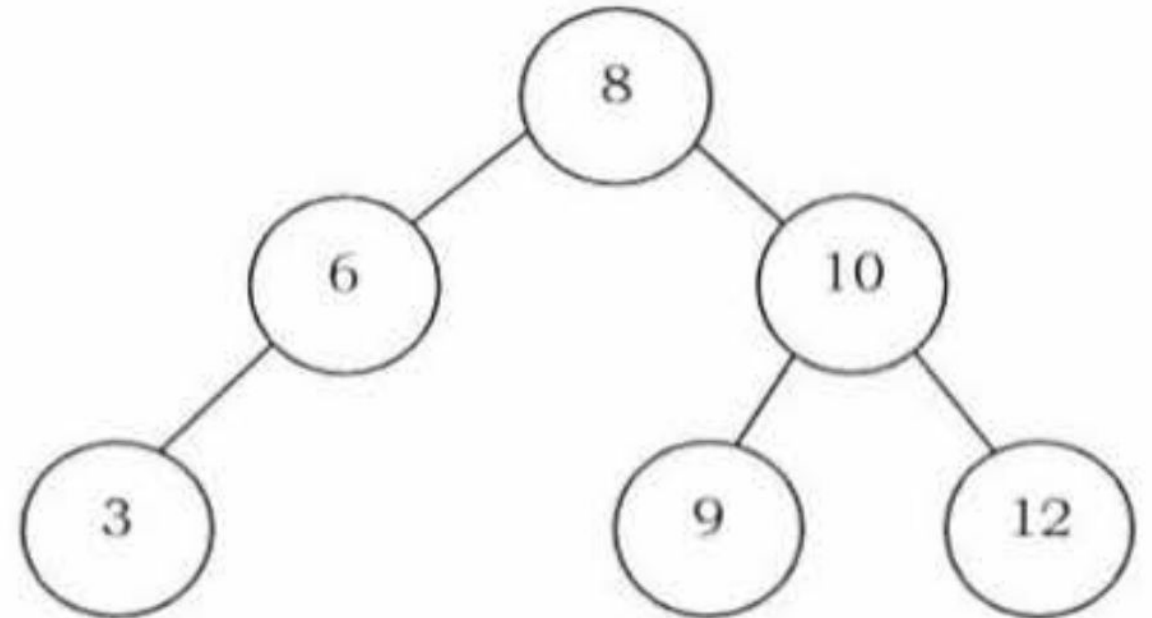
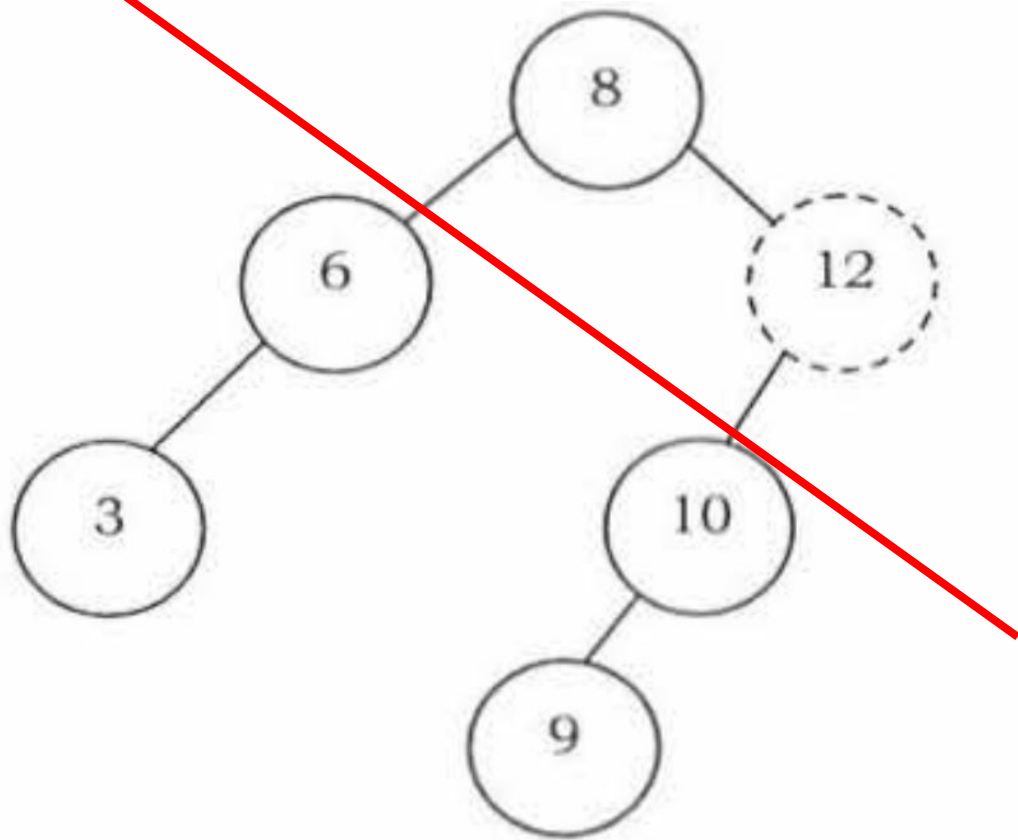
Apa itu AVL?

- Dalam Height-balanced $HB(k)$, jika $k=1$ (jika factor seimbang adalah 1) seperti pada suatu BST disebut suatu AVL tree. Artinya AVL tree adalah suatu BST dengan kondisi seimbang: perbedaan tinggi subtree kiri dengan tinggi subtree kanan paling besar adalah 1.

Properti dari AVL tree

- Suatu tree dikatakan AVL jika:
 - Suatu BST
 - Untuk beberapa node X , perbedaan tinggi subtree kiri dari X dan subtree kanan dari X adalah paling banyak 1

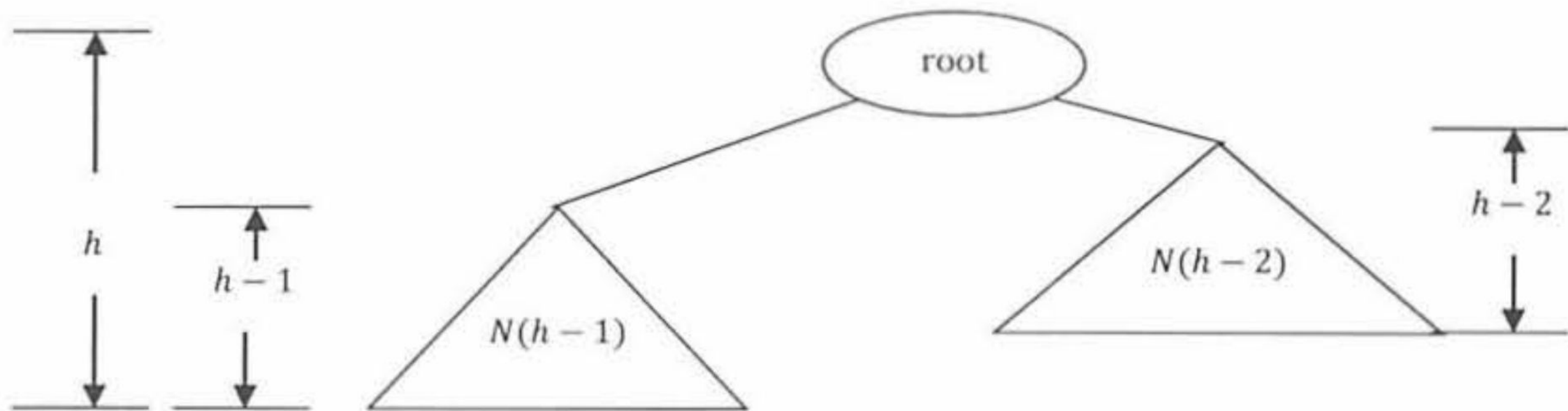
AVL tree



Jumlah node minimum dan maksimum

- Apabila kita asumsikan jumlah tinggi dari AVL tree adalah h dan $N(h)$ mengindikasikan jumlah dari Node dengan tinggi h . untuk mendapatkan jumlah node dengan tinggi h , kita harus mengisi tree dengan jumlah node yang mungkin. Dengan demikian jika kita mengisi subtree kiri dengan tinggi $h-1$ maka kita seharusnya mengisi subtree kanan dengan tinggi $h-2$. hasilnya jumlah minimum node dengan tinggi h adalah:
- $N(h) = N(h - 1) + N(h - 2) + 1$

- $N(h - 1)$ mengindikasikan jumlah minimum dari node dengan tinggi $h - 1$.
- $N(h - 2)$ mengindikasikan jumlah minimum dari node dengan tinggi $h - 2$.
- Dalam ekspresi sebelumnya, "1" mengindikasikan node saat ini.



Deklarasi AVL tree

- Deklarasi AVL tree mirip dengan BST. Akan tetapi untuk menyederhanakan operasi, kita menginputkan tinggi sebagai bagian dari deklarasi.

```
class AVLNode:  
    def __init__(self,data,balanceFactor,left,right):  
        self.data= data  
        self.balanceFactor = 0  
        self.left = left  
        self.right= right
```


Mencari tinggi dari AVL tree

```
def height(self):  
    return self.recHeight(self.root)  
  
def recHeight(self, root):  
    if root == None:  
        return 0  
    else:  
        leftH =self.recHeight(r.left)  
        rightH = self.recHeight(r.right)  
        if leftH>rightH:  
            return 1 + leftH  
        else:  
            return 1+ rightH
```

Rotasi

- Jika struktur tree berubah (karena insertion atau deletion), kita harus mengubah tree untuk mengembalikan property dari AVL. Ini bisa dilakukan dengan single rotation atau multi rotation. Selama suatu insertion atau deletion melibatkan menambah/menghapus suatu node tunggal, ini bisa meningkatkan/menurunkan tinggi dari subtree oleh 1.
- Jika properti AVL tree dari dilanggar pada suatu node X, yang artinya perbedaan tinggi dari $\text{left}(X)$ dan $\text{right}(X)$ tepatnya 2. Ini karena jika kita menyeimbangkan AVL tree setiap waktu, maka pada beberapa point, perbedaan tinggi $\text{left}(X)$ dan $\text{right}(X)$ tepatnya 2.
- Rotasi ini adalah Teknik untuk mengembalikan property dari AVL tree. Artinya kita perlu mengaplikasikannya pada node X.

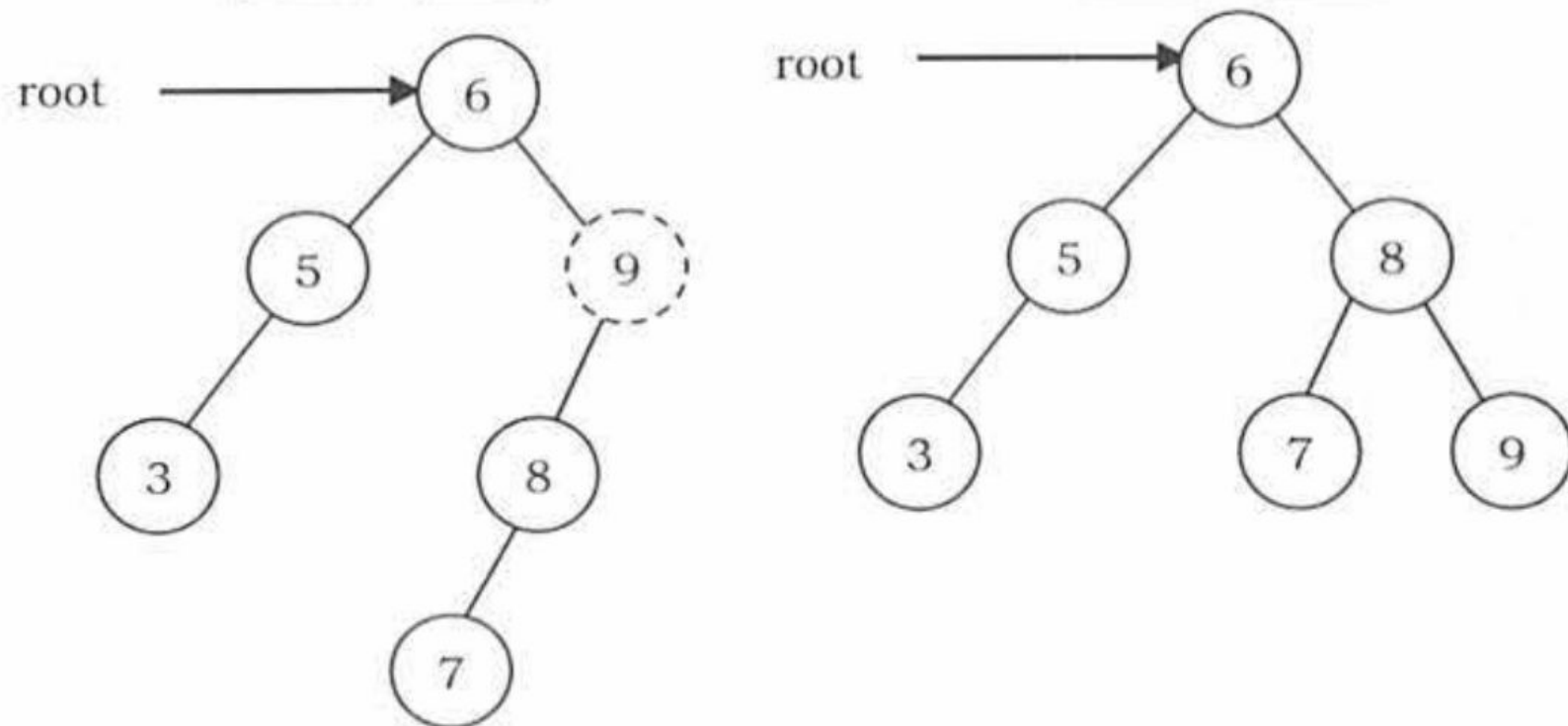
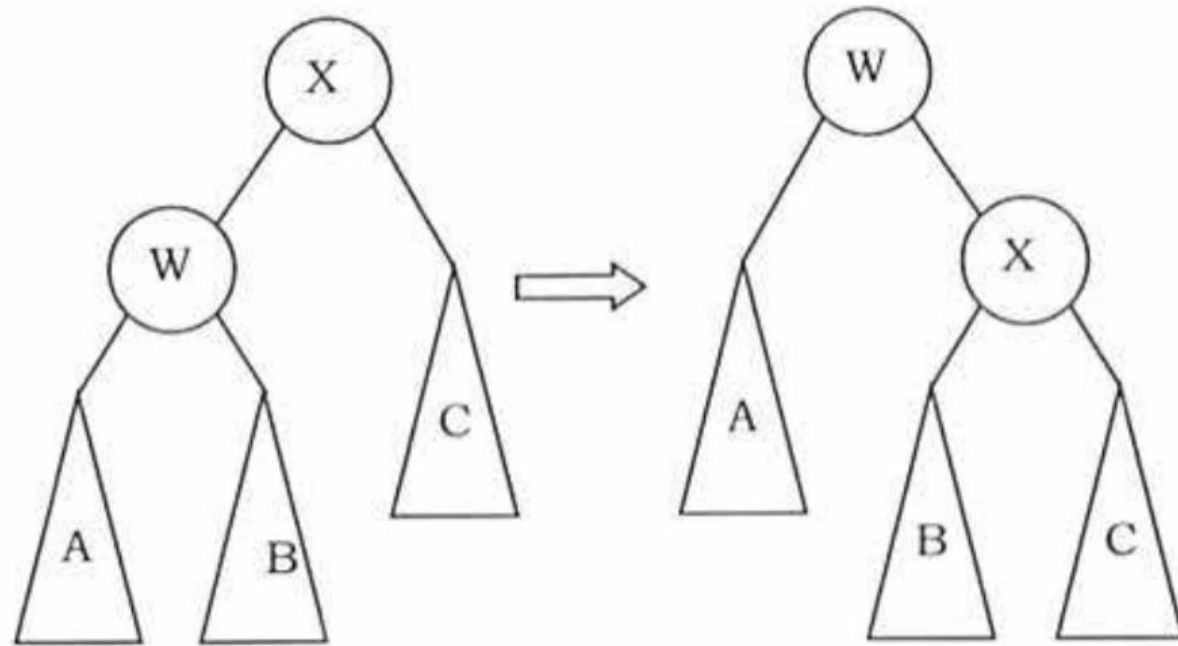
Tipe-tipe pelanggaran

- Kita asumsikan node tersebut adalah X. selama beberapa node memiliki paling banyak dua anak, dan suatu tinggi yang tidak seimbang yaitu dua tinggi subtree X memiliki selisih 2.
1. Penyisipan ke subtree kiri anak kiri X.
 2. Penyisipan ke subtree kanan anak kiri X.
 3. Penyisipan ke subtree kiri anak kanan X.
 4. Penyisipan ke subtree kanan anak kanan X

- Kasus 1 dan 4 simetris dan mudah diselesaikan dengan rotasi tunggal. Demikian pula, kasus 2 dan 3 juga simetris dan dapat diselesaikan dengan rotasi ganda (perlu dua rotasi tunggal).

Single Rotation

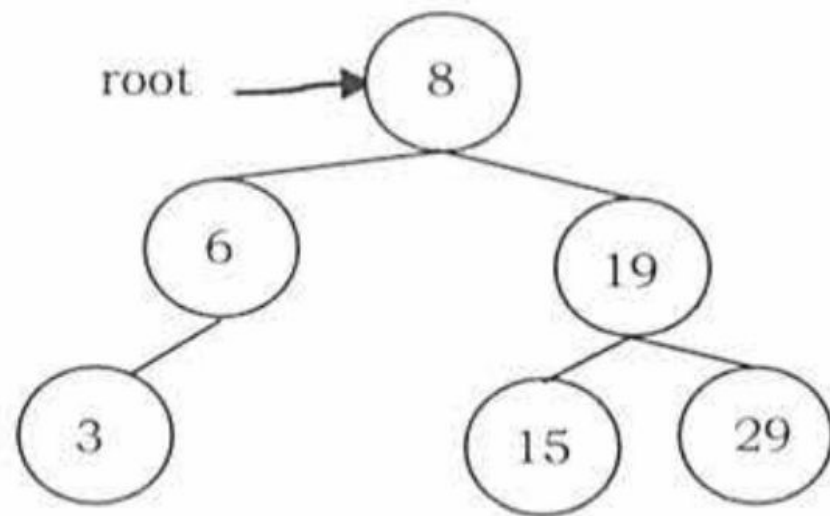
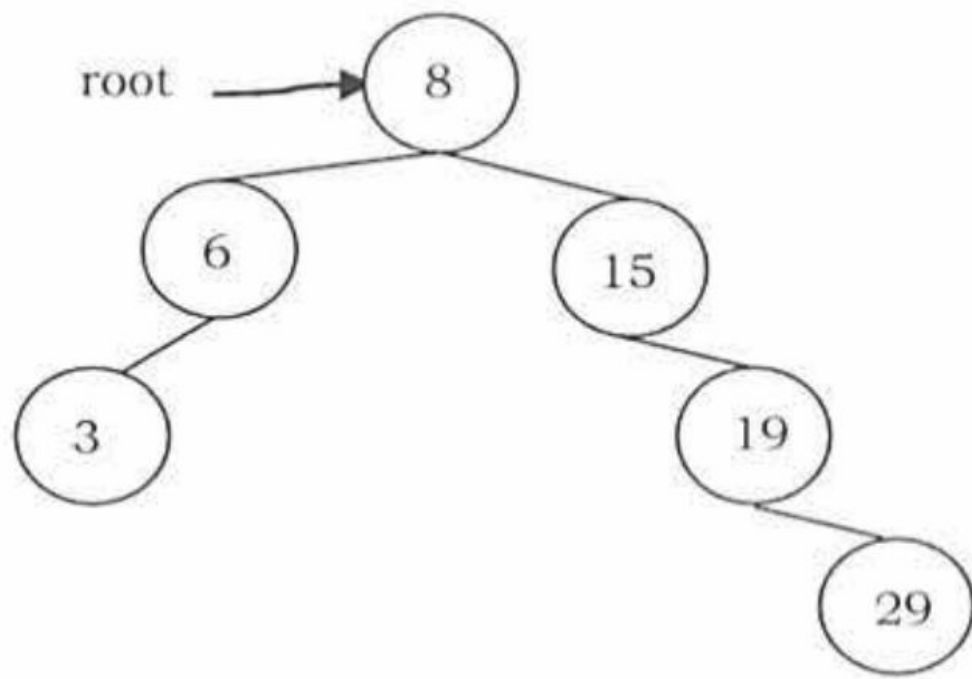
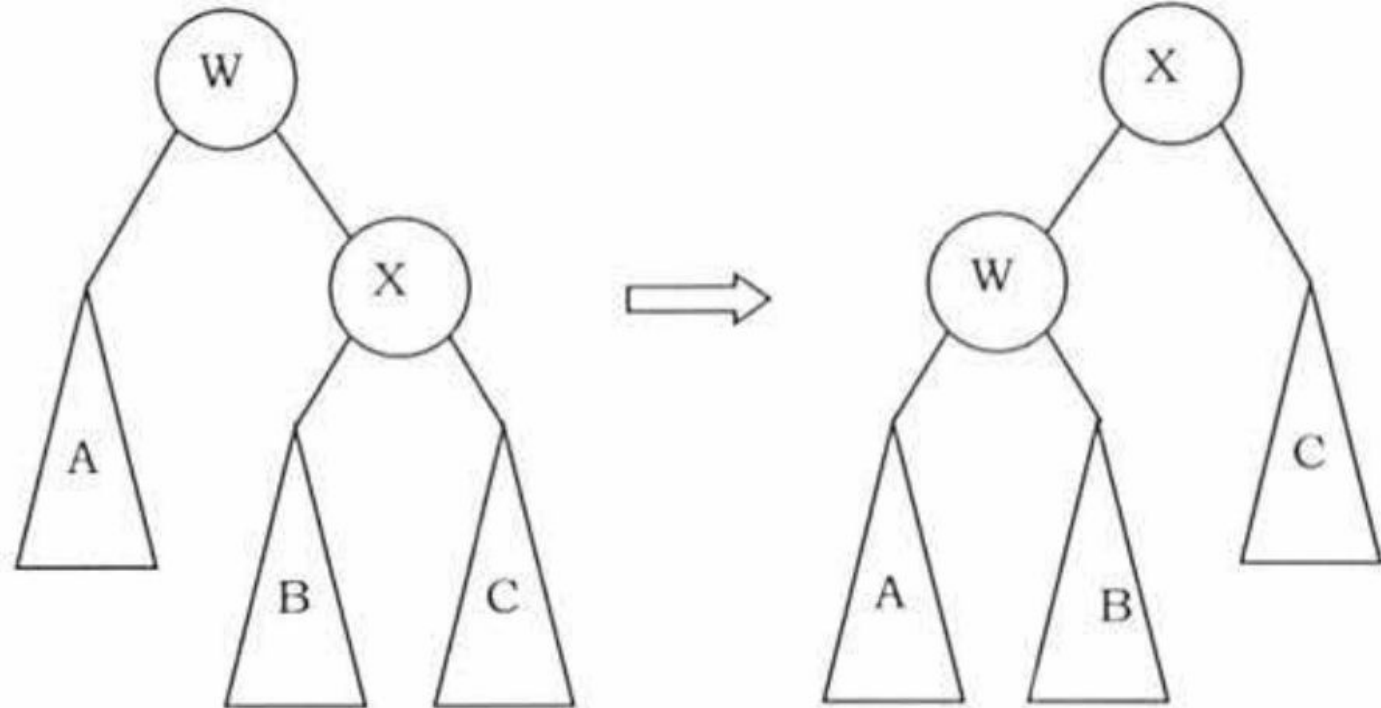
- Left Left Rotation (LL Rotations [Case-1]):
- Dalam kasus di bawah ini, simpul X tidak memuaskan properti pohon AVL. Sebagai dibahas sebelumnya, rotasi dokumen tidak harus dilakukan pada akar pohon. Secara umum, kita mulai dari node dimasukkan dan berjalan menaiki pohon, memperbarui informasi saldo di setiap node di jalan.



- Sebagai contoh, pada gambar sebelumnya, setelah memasukkan 7 di pohon AVL asli di sebelah kiri, simpul 9 menjadi tidak seimbang. Jadi, kita melakukan rotasi left-left tunggal pada angka 9. Sebagai hasilnya kita mendapatkan pohon di kanan.

```
def singleLeftRotate(self, root):  
    W = root.left  
    root.left = W.right  
    W.right = root  
    return W
```

Rotasi Kanan Kanan (Rotasi RR) (Kasus-4): Dalam kasus ini, simpul X tidak memuaskan properti pohon AVL.

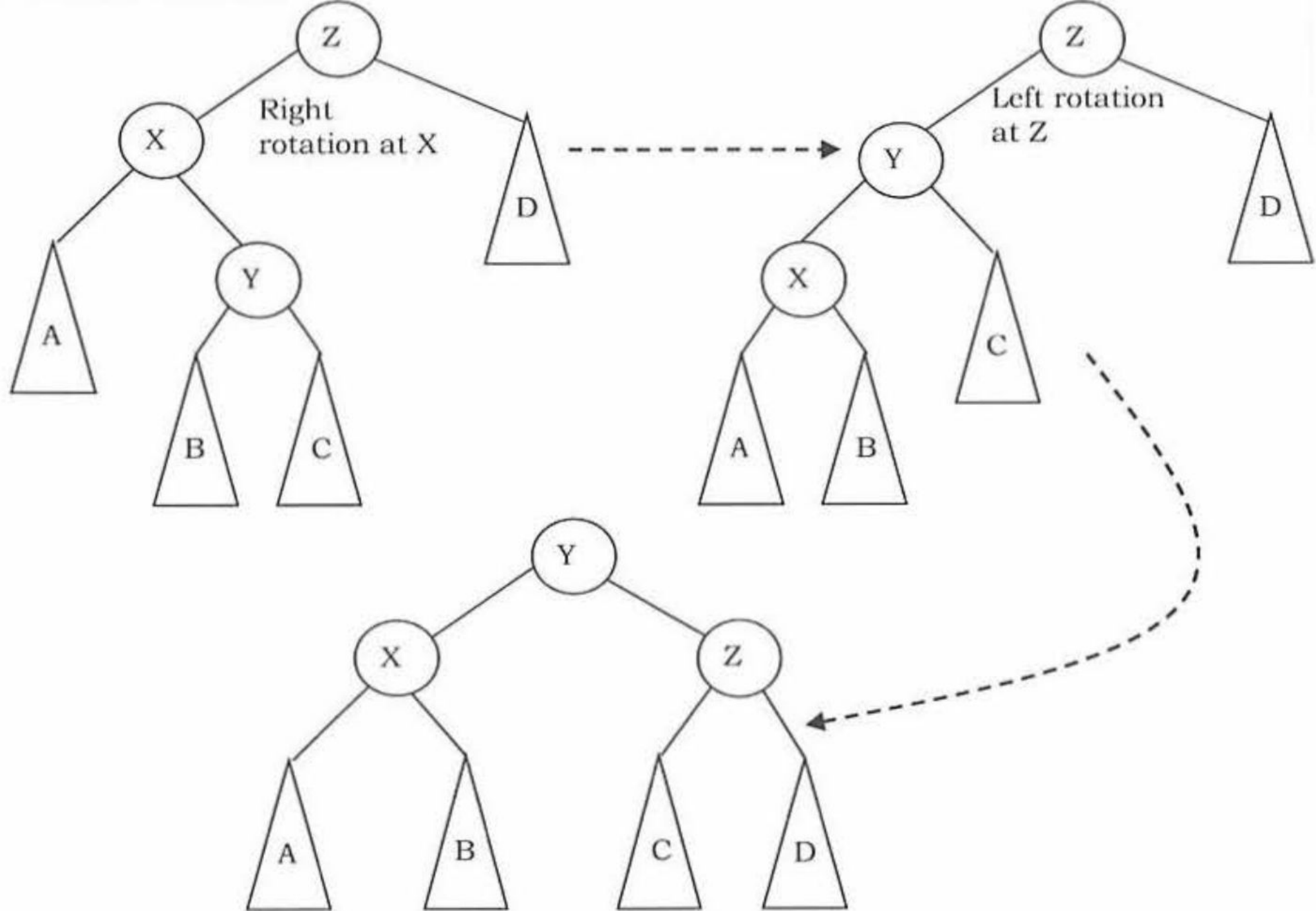


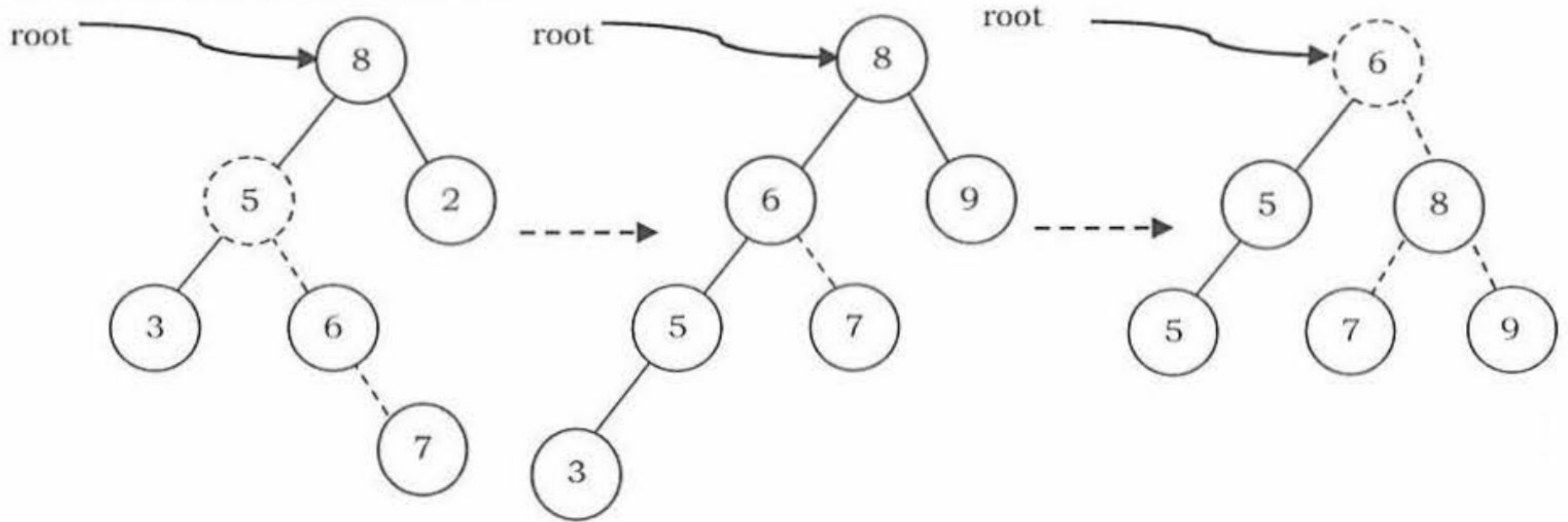
- Misalnya, pada gambar di atas, setelah penyisipan 29 di pohon AVL asli di sebelah kiri, simpul 15 menjadi tidak seimbang. Jadi, kami melakukan rotasi kanan-kanan tunggal pada 15. Akibatnya, kami mendapatkan pohon di kanan.

```
def singleRightRotate(self, root):  
    X = root.right  
    root.right = X.left  
    X.left = root  
    return X
```

Double Rotation

- Kiri Kanan Rotasi (Rotasi LR) (Kasus-2): Untuk kasus-2 dan kasus-3 rotasi tunggal tidak memperbaiki masalah. Kita perlu melakukan dua rotasi.

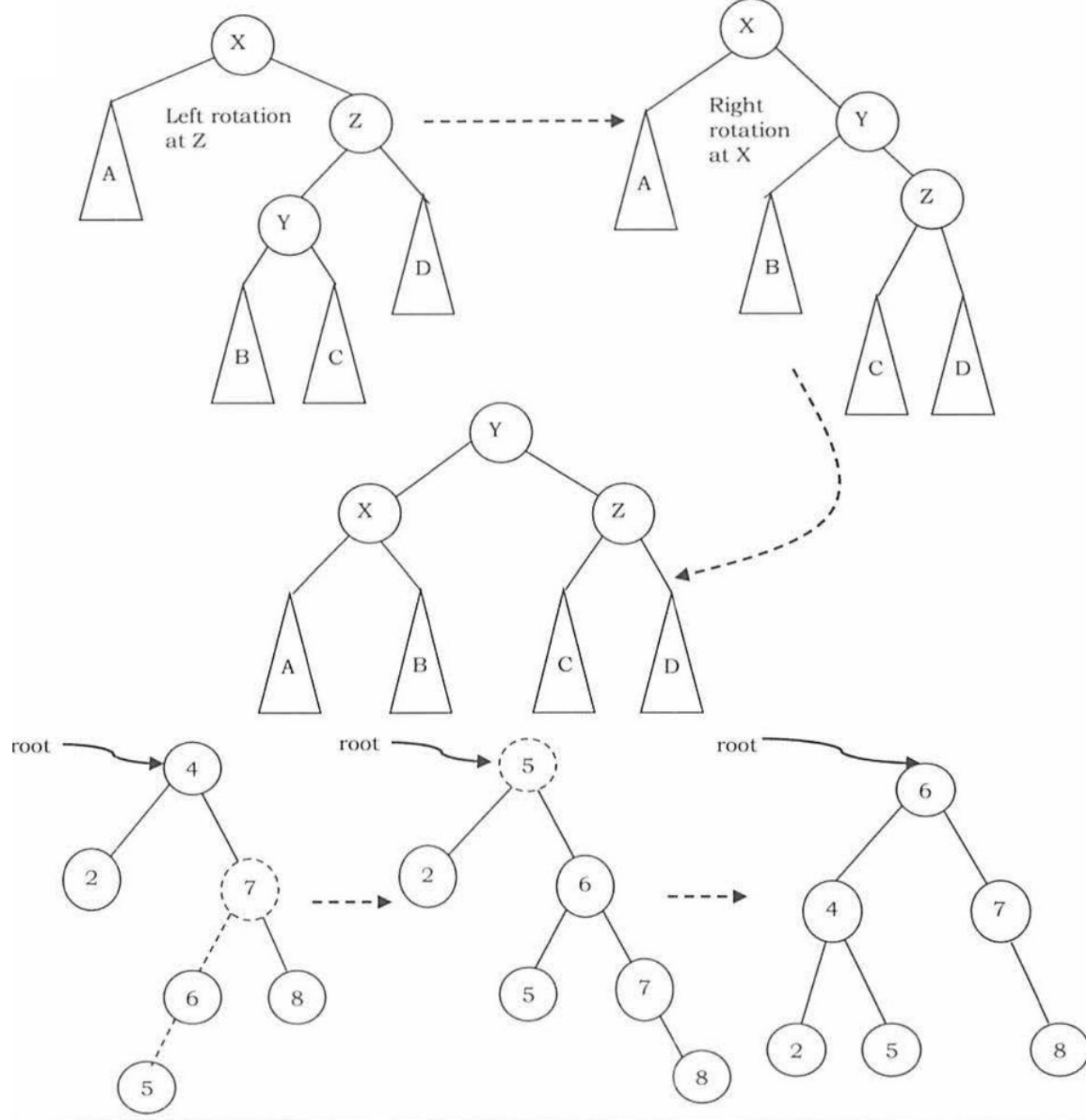




sebagai contoh, mari kita perhatikan susunan pohon berikut: Penyisipan 7 membuat skenario kasus-2 dan sisi kanan pohon adalah satu setelah rotasi ganda.

```
def rightLeftRotate(self, root):
    X = root.left
    if X.balanceFactor == - 1:
        root.balanceFactor = 0
        X.balanceFactor = 0
        root = self.singleLeftRotate(root)
    else:
        Y = X.right
        if Y.balanceFactor == 1:
            root.balanceFactor = 1
            X.balanceFactor = 0
        elif Y.balanceFactor == 0:
            root.balanceFactor = 0
            X.balanceFactor = 0
        else:
            root.balanceFactor = 0
            X.balanceFactor = -1
        Y.balanceFactor = 0
        root.left = self.singleRightRotate(X)
        root = self.singleLeftRotate(root)
    return root
```

- Rotasi Kiri Kanan (Rotasi RL) (Kasus -3): Mirip dengan kasus-2, kita perlu melakukan dua hubungan untuk memperbaikinya skenario.



Sebagai contoh, mari kita perhatikan pohon berikut: Penyisipan 5 adalah membuat skenario kasus-3 dan kanan pohon samping adalah yang setelah rotasi ganda.


```
def rightLeftRotate(self, root):
    X = root.right
    if X.balanceFactor == 1:
        root.balanceFactor = 0
        X.balanceFactor = 0
        root = self.singleRightRotate(r)
    else:
        Y = X.left
        if Y.balanceFactor == -1:
            root.balanceFactor = 0
            X.balanceFactor = 1
        elif Y.balanceFactor == 0:
            root.balanceFactor = 0
            X.balanceFactor = 0
        else:
            root.balanceFactor = -1
            X.balanceFactor = 0
        Y.balanceFactor = 0
        root.right == self.singleLeftRotate(X)
        root = self.singleRightRoate(root)
    return root
```

Insertion ke suatu AVL tree

- Penyisipan ke pohon AVL mirip dengan penyisipan BST. Setelah memasukkan elemen, kita hanya perlu memeriksa apakah ada ketidakseimbangan ketinggian. jika ada ketidakseimbangan, panggil fungsi rotasi yang sesuai.

```

def insert(self,data):
    newNode = AVLNode(data,O,None,None)
    [self.root,taller] = self.reInsertAVL(self.root, newNode)

def reInsertAVL(self, root, newNode):
    if root == None:
        root = newNode
        root. balanceFactor = 0
        taller = True
    elif newNode.data < root.data:
        [root.left, taller] = self.reInsertAVL(root.left, newNode)
        if taller:
            if root. balanceFactor == 0 :
                root. balanceFactor = -1
            elif root.balanceFactor == 1:
                root. balanceFactor= 0
                taller = False

            else:
                root= self.rightLeftRotate(root)
                taller = False
    else :
        [root.right,taller] = self.reInsertAVL(root. right, newNode)
        if taller:
            if root.balanceFactor == -1:
                root.balanceFaclor = 0
                taller = False
            elif root.balanceFa,ctor == 0 :
                root.balanceFactor = 1
            else:
                root = self.rightLeftRotate(root)
                taller = False

    return [root,taller]

```

Implementasi Lengkap

```
class AVLNode:
    def __init__(self, data, balanceFactor, left, right):
        self.data = data
        self.balanceFactor = 0
        self.left = left
        self.right = right

class AVLtree:
    def __init__(self):
        self.root = None

    def inOrderPrint(self):
        self.reclnOrderPrint(self.root)

    def reclnOrderPrint(self, root):
        if root != None:
            self.reclnOrderPrint(root.lcft)
            print root.data
            self.reclnOrderPrint(root.right)

    def insert(self,data):
        newNode =AVLNode(data,0,None,None)
        [self.root,taller] = self.reclnInsertAVL(self.root,newNode)
```

```
def reInsertAVL(self, root, newNode):
    if root == None:
        root = newNode
        root.balanceFactor = 0
        taller = True
    elif newNode.data < root.data:
        [root.left,taller] = self.reInsertAVL(root.left, newNode)
        if taller:
            if root.balanceFactor == 0:
                root.balanceFactor = -1
            elif root.balanceFactor == 1:
                root.balanceFactor = 0
                taller = False
            else:
                root = self.rightLeftRotate(root)
                taller = False
    else :
        [root.right,taller] = self.reInsertAVL(root.right, newNode)
        if taller:
            if root.balanceFactor == -1:
                root.balanceFactor = 0
                taller = False
            elif root.balanceFactor == 0:
                root.balanceFactor = 1
            else:
                root = self.rightLeftRotate(root)
                taller = False
    return [root,taller]
```

```
def rightLeftRotate(self, root):
    X = root.right
    if X.balanceFactor == 1:
        root.balanceFactor = 0
        X.balanceFactor = 0
        root = self.singleRightRotate(root)
    else:
        Y = X.left
        if Y.balanceFactor == -1 :
            root.balanceFactor = 0
            X.balanceFactor == 1
        elif Y.balanceFactor == 0:
            root.balanceFactor = 0
            X.balanceFactor = 0
        else:
            root.balanceFactor = -1
            X.balanceFactor = 0
        Y.balanceFactor = 0
        root.right = self.singleLeftRotate(X)
        root = self.singleRightRotate(root)
    return root
```

```
def rightLeftRotate(self,root):
    X = root.left
    if X.balanceFactor == -1:
        root.balanceFactor = 0
        X.balanceFactor = 0
        root = self.singleLeftRotate(root)
    else:
        Y = X.right
        if Y.balanceFactor == -1:
            root.balanceFactor = 1
            X.balanceFactor = 0
        elif Y.balanceFactor == 0:
            root.balanceFactor = 0
            X.balanceFactor = 0
        else:
            root.balanceFactor = 0
            X.balanceFactor = -1
        Y.balanceFactor = 0
        root.left = self.singleRightRoate(X)
        root = self.singleLeftRotate(root)
    return root
```

```
def singleRightRotate(self, root):
    X = root.right
    root.right = X.left
    X.left = r
    return X
```

```
def singleLeftRotate(self,root):
    W = root.left
    root.left= W.right
    W.right = root
    return W
```

```
def height(self):
    return self.recHeight(self. root)
```

```
def recHeight(self,root):
    if root == None:
        return 0
    else:
        leftH = self.recHeight(root.left)
        rightH = self.recHeight(root.right)
        if leftH>rightH:
            return 1 + leftH
        else:
            return 1 + rightH
```

```
def tester():
    avl = AVLTree()
    data = [3,1,9,6,0, 11,2,5,4]
    for i in range(len(data)):
        avl.insert(data[i])
    avl.inOrderPrint()
    print ("height = ",avl.height())
```

```
if __name__ == '__main__':
    tester()
```


- Referensi:
- Karumachi N.,2016. Data Structure and Algorithmic Thinking with Python